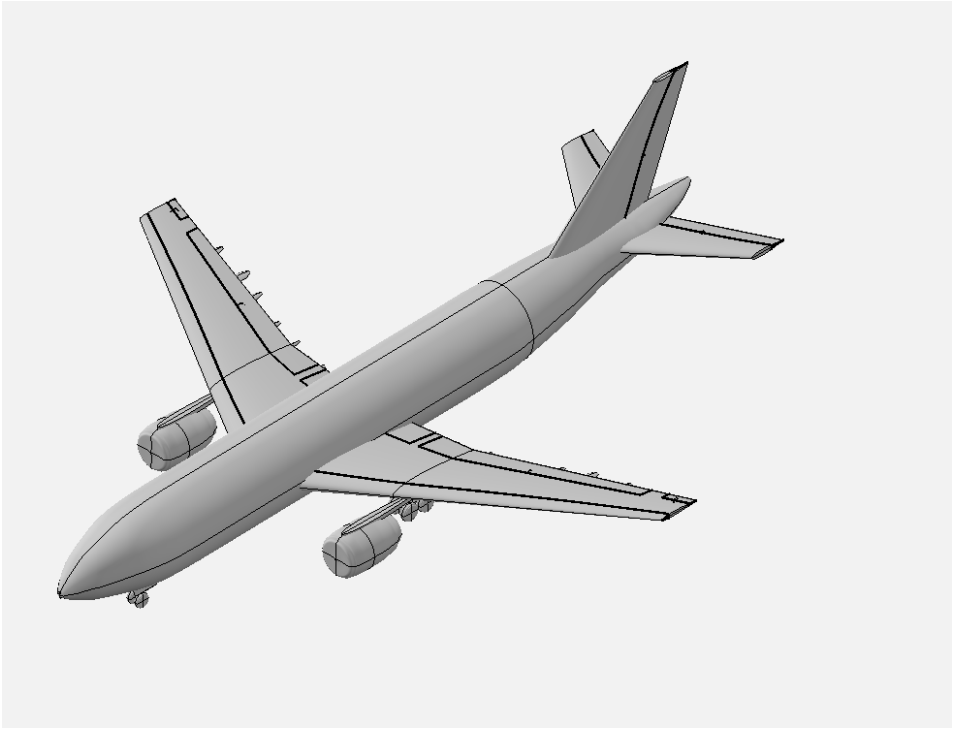
Guía de uso de aircraft_recon.py

Reconstrucción paramétrica de aviones en OpenVSP 3.53.1



Modelo del A300-600 construido con el programa, abierto en OpenVSP.

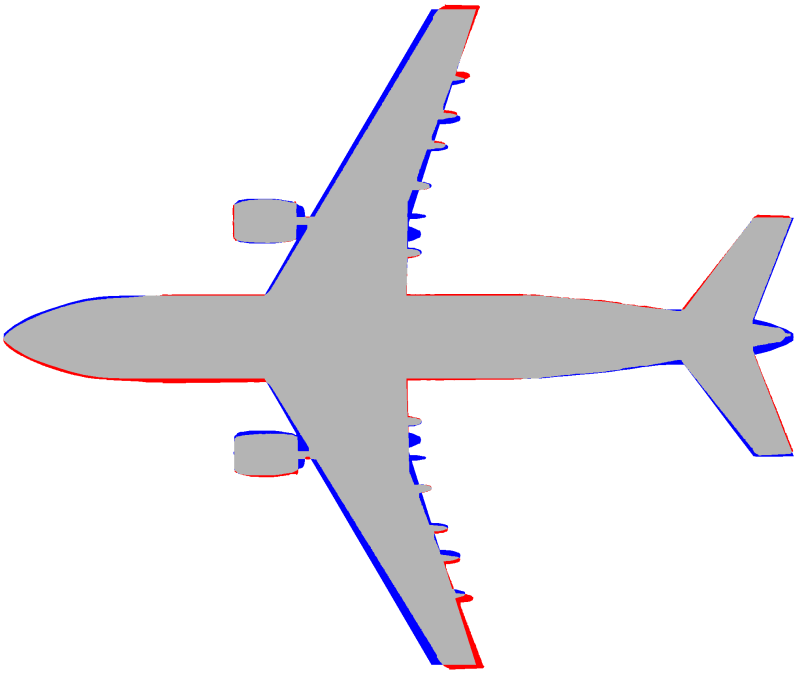
Contenido

1. Qué hace el programa 2
2. Instalación 2
3. Proyectos y versiones 3
4. Avión desde archivos, sin imágenes 3
5. Avión desde tres vistas 4
6. Revisar el resultado 5
7. Cambiar parámetros y deshacer 5
8. Definir un ala por secciones 6
9. Analizar el modelo en OpenVSP 6
10. Problemas frecuentes 7
11. Referencia de comandos 7

1. Qué hace el programa

aircraft_recon.py crea un modelo de OpenVSP de un avión y lo compara con sus planos. Trabaja de dos formas: a partir de tres vistas dibujadas (planta, perfil y frontal) o a partir de archivos con las medidas.

Su idea central es separar lo que se mide de lo que se ajusta. Lo que el dibujo permite medir, como las estaciones del fuselaje, se mide. El resto lo ajusta un optimizador con pocos parámetros cada vez. Las cotas que marques como fiables nunca se modifican.



Resultado de compare en el A300. Rojo: solo en el dibujo. Azul: solo en el modelo. Gris: coinciden.

Qué no hace

- No deduce perfiles, grosores ni secciones a partir de tres siluetas: son valores supuestos.
- No calcula aerodinámica. Eso se hace en OpenVSP y VSPAERO con el modelo que genera (sección 9).
- Construye perfiles simétricos, sin curvatura.

2. Instalación

Necesitas Windows, Git y Python 3.13, que es la versión para la que está compilado OpenVSP.

1. Descarga el programa:

```
git clone https://github.com/IIAlbertyII/OpenVSPAuto.git
cd OpenVSPAuto
```

2. Descarga OpenVSP para Windows desde openvsp.org/download.php y descomprímelo dentro de `OpenVSPAuto`, en una carpeta cuyo nombre empiece por `OpenVSP`. El programa se ha probado con la

3.53.1.

3. Crea el entorno. Después, en cada sesión basta con activarlo.

```
python -m venv C:\venvs\vsp313
C:\venvs\vsp313\Scripts\activate
pip install numpy opencv-python scipy
```

4. Comprueba que todo funciona desde la carpeta del programa:

```
python aircraft_recon.py selftest
    SELFTEST OK
python aircraft_recon.py check-env
    Python 3.13.16 64bit
    OpenVSP OK, version: OpenVSP 3.53.1
```

Rutas: usa rutas sin acentos ni «ñ» para los archivos temporales. Si OpenVSP está en otro sitio, indícalo con la variable VSP_PYTHON.

3. Proyectos y versiones

Cada avión es un **proyecto** en su propia carpeta, `aircraft/NOMBRE/`, de modo que nada se mezcla entre modelos. Cada cambio crea una **versión** numerada que se puede recuperar.

Comando	Para qué
`project-new NOMBRE`	Crea un proyecto y lo selecciona
`project-list`	Lista los proyectos; * marca el actual
`project-use NOMBRE`	Selecciona un proyecto
`project-copy A B`	Copia un proyecto, sin versiones salvo con --full
`--project NOMBRE comando`	Usa otro proyecto solo para ese comando; va antes del comando

Dentro de cada proyecto, `versions/` guarda los `.vsp3`, `renders/` las comparaciones y `processed/` las máscaras. **El modelo que abres en OpenVSP es el .vsp3 de la versión actual.**

La carpeta antigua `project/` contiene el A300 original y se usa cuando no hay proyecto seleccionado. Consulta el proyecto activo en la línea `[project: ...]` que imprime cada comando.

4. Avión desde archivos, sin imágenes

Es el camino más rápido si ya tienes las medidas: un JSON de parámetros, otro de fuselaje y otro de ala. Este ejemplo crea un A340; sus archivos están en la carpeta de trabajo del autor, no en el repositorio:

```
python aircraft_recon.py project-new demo340 ^
  --params   aircraft\a340\fuentes\a340_params.json ^
  --fuselage aircraft\a340\fuentes\a340_fuselage.json ^
  --wing     aircraft\a340\fuentes\a340_wing.json
python aircraft_recon.py build
    -> version 1
python aircraft_recon.py wing-check
```

`wing-check` lee cada sección del ala en OpenVSP y muestra lo esperado frente a lo obtenido. Los dos números de cada parámetro deben coincidir:

```
sec 1  Span 11.061->11.061  Root_Chord 10.550->10.550  Tip_Chord 6.900->6.900
planform span 58.00 m (total with winglet 60.30 m), area 361.2 m2
```

El resultado queda en `aircraft\demo340\versions\v001.vsp3`.

5. Avión desde tres vistas

Prepara tres imágenes con el morro a la izquierda y anota una longitud real de cada una: la longitud total en planta y perfil, y la envergadura en la frontal.

Paso 1. Crear el proyecto y calibrar

```
python aircraft_recon.py project-new MI_AVION
python aircraft_recon.py calibrate --view top   --image planta.png  --length 53.85 --keep-parts
python aircraft_recon.py calibrate --view side  --image perfil.png  --length 54.08 --keep-parts
python aircraft_recon.py calibrate --view front --image frontal.png --length 44.84 --keep-parts
```

Abre `processed/*_mask.png` y comprueba la silueta. Si te equivocaste en la escala, `rescale --view V --length L` la corrige sin rehacer la máscara.

Paso 2. Limpiar las máscaras

Quita motores y tren con `erase-mask`, y vuelve a poner solo los motores con `restore-mask`. Se abre una ventana para marcar rectángulos con el ratón. Así se crean las variantes **core** (sin motores) y **extras** (con motores) que usa `auto`.

```
python aircraft_recon.py erase-mask --view top
python aircraft_recon.py restore-mask --view top
```

Paso 3. Fijar las medidas conocidas

Da el diámetro real del fuselaje. Sin esto, `fuselage-fit` se detiene porque los valores de plantilla no encajan con la máscara. En `params.json`, marca las cotas fiables con `"fixed": true`.

```
python aircraft_recon.py set fuselage.max_width 5.64 --force
python aircraft_recon.py set fuselage.max_height 5.64 --force
```

Paso 4. Reconstruir

```
python aircraft_recon.py auto --step --masks
```

Con `--step`, el programa para tras cada paso y pregunta `[Enter] continuar | [u] deshacer | [q] salir`. Ejecútalo en una consola interactiva. Las fases son:

Fase	Qué construye	Máscara
`airframe`	Fuselaje, ala, colas, diedros y grosores	core
`extras`	Góndolas, pilones y carenados	extras
`finish`	Tren, superficies de mando, ventanas y malla fina	—

Puedes ejecutar una sola fase con `auto --phase airframe`. Si una etapa empeora el error más de un 2 %, se revierte sola. Para un avión nuevo, `wizard` te pregunta los datos y lanza estos pasos.

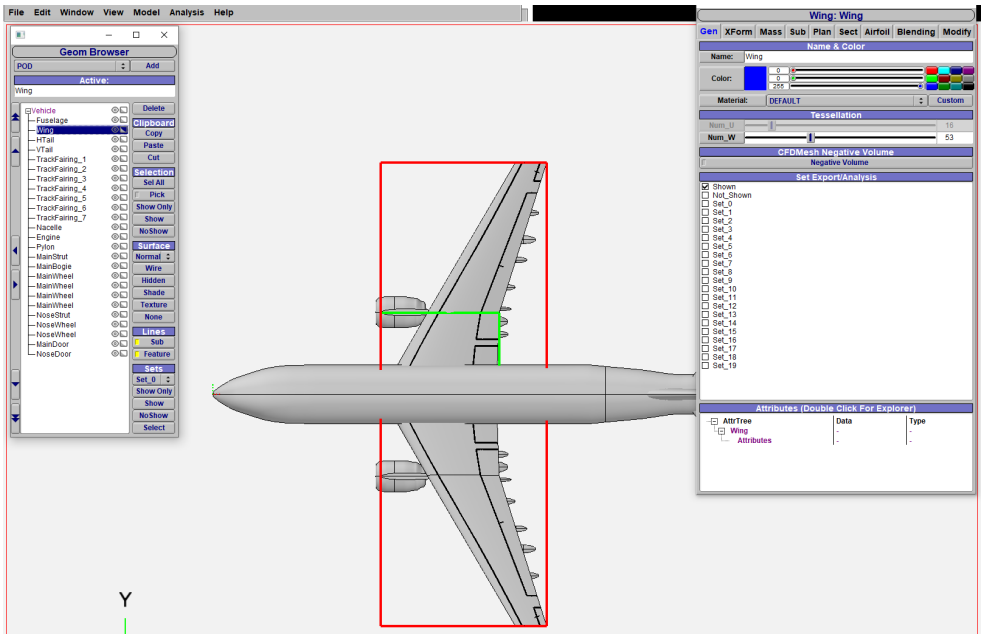
6. Revisar el resultado

```
python aircraft_recon.py compare
python aircraft_recon.py report-metrics
```

`compare` escribe imágenes en `renders/`. `report-metrics` da una línea por vista:

Métrica	Significado	A300 (v088)
IoU	Solape de siluetas. 1 es coincidencia perfecta	Planta 0.92, perfil 0.92, frontal 0.75
Chamfer	Distancia media entre contornos	0.15–0.17 m
H95	Distancia que supera solo el 5 % de los puntos del contorno	0.43–0.58 m

Para verlo en 3D, abre el `.vsp3` de la versión actual con `vsp.exe`. Atajos útiles: F5 planta, F6 frontal, F7 perfil, F8 isométrica, F para encajar. En el Geom Browser, **Sel All** y **Shade** sombrean el modelo.



El mismo A300 en OpenVSP, con el ala seleccionada y su panel de parámetros a la derecha.

7. Cambiar parámetros y deshacer

Comando	Efecto
`set wing.dihedral 5.0`	Fija un valor y crea una versión
`delta wing.x -0.3`	Suma o resta al valor actual
`patch cambios.json`	Aplica varios cambios con su motivo
`history --last 5`	Muestra las últimas versiones
`diff` o `diff 3 7`	Muestra qué cambió entre versiones
`undo`	Vuelve a la versión anterior
`checkout 18`	Restaura una versión concreta
`optimize --free wing.sweep wing.x`	Ajusta solo esos parámetros contra las siluetas

Un parámetro `measured` o `fixed` se rechaza salvo con `--force`. Ejemplo de archivo de parche:

```
[{"operation":"modify","parameter":"wing.x","delta":-0.3,
  "reason":"ala 300 mm demasiado adelantada"}]
```

8. Definir un ala por secciones

El archivo `wing.json` describe una semiala en metros y grados. Cada estación lleva su posición, cuerda, borde de ataque, altura, espesor y twist:

```
{ "twist_mode": "absolute", "u_mode": "section",
  "stations": [
    {"y": 0.0, "chord": 10.27, "x_le": 19.36, "z": -1.27,
     "thickness": 0.135, "twist": -0.63},
    {"y": 8.5, "chord": 5.00, "x_le": 24.81, "z": -0.53,
     "thickness": 0.11, "twist": -1.03} ],
  "controls": [
    {"name": "Aileron", "y0": 18.8, "y1": 23.6, "chord_frac": 0.24, "le": false} ] }
```

Comando	Para qué
`wing-fit`	Mide el ala de la máscara de planta y escribe wing.json
`wing-stations --load F.json`	Instala un ala en el proyecto
`wing-check`	Compara lo esperado con lo leído de OpenVSP
`wing-export F.json --fuselage`	Crea un .vsp3 solo con esa ala

Twist: el valor de cada sección es absoluto por defecto; OpenVSP no los acumula. Los mandos usan `u_mode: "span"` por defecto; en los modelos de este proyecto se usa `section`. Comprueba siempre con `wing-check`.

9. Analizar el modelo en OpenVSP

El programa no incluye un comando de análisis. Haz estos pasos en OpenVSP sobre una **copia** del `.vsp3`, porque VSPAERO escribe sus archivos junto al modelo.

- **Resistencia parásita** (Analysis > Parasite Drag): elige el ala como **Ref. Wing**. Excluye el tren en crucero: las ruedas tienen factores de forma enormes. En el A300, con el tren incluido y un error de altitud, el CD0 salió 0.20 en lugar de 0.0165.
- **Comprueba las unidades de altitud:** en una prueba, 35 000 pies se interpretaron como metros.
- **VSPAERO:** para la polar usa las superficies sustentadoras en modo vórtice (VLM). Como los perfiles son simétricos, el CL es 0 a 0° de ángulo de ataque.

10. Problemas frecuentes

Lo que ves	Causa y solución
`fuselage.max_width/max_height ... are far from what the masks show`	El diámetro sigue con el valor de plantilla o la máscara tiene motores y cotas. Limpia la máscara y fija el diámetro (sección 5).
`EOFError: EOF when reading a line` en auto o wizard	Faltan las máscaras core y extras y el programa quiere preguntar. Ejecútalo en una consola interactiva.
`active masks: core for no view`	Aún no existen máscaras core. Usa erase-mask y restore-mask.
`no fairing-sized blobs under the wing`	La vista frontal no muestra carenados de ese tamaño. Revisa la máscara.
`no reliable improvement; parameters unchanged`	No es un error: el optimizador no mejoró. Prueba más iteraciones (--iters) o revisa las máscaras.
`UnicodeEncodeError` en smooth	Fallo conocido al redirigir la salida. Ejecuta antes `set PYTHONUTF8=1`.
OpenVSP no se importa o falla el binding	No estás en el entorno de Python 3.13. Activa C:\venvs\vsp313 y ejecuta check-env.
El ala cambia de tamaño al reabrir el .vsp3	Área de sección obsoleta. Usa wing-check; el programa ya hace Update tras cada sección.

11. Referencia de comandos

Sí indica que se ejecutó en OpenVSP 3.53.1 en esta máquina durante la preparación de esta guía, en una copia aislada. **No** indica que no se ejecutó en esa prueba, por ser interactivo o largo.

Grupo	Comando	Probado
Diagnóstico	`selftest, check-env, inspect, timing, selfcheck, probe-subsurf`	Sí
Diagnóstico	`probe, probe-geom, check-var-comp`	No
Proyecto	`project-new, project-list, project-use, project-copy, migrate`	Sí
Proyecto	`project-adopt, init, wizard`	No
Máscaras	`calibrate, masks, landmarks`	Sí
Máscaras	`erase-mask, restore-mask, rescale`	No

Grupo	Comando	Probado
Fuselaje	`fuselage-fit, simplify`	Sí
Fuselaje	`smooth`	Falla al redirigir la salida
Fuselaje	`fuselage-test, fuselage-simplify`	No
Ala	`wing-fit, wing-stations, wing-check, wing-export`	Sí
Motores y carenados	`nacelle-fit, fairings-fit`	Sí
Optimización	`optimize, front-fit, auto --phase airframe`	Sí
Optimización	`auto completo (--step, --masks, tres fases)`	No
Construir y medir	`build, compare, report-metrics`	Sí
Versiones	`set, delta, patch, history, diff, undo, checkout`	Sí

La lista completa de opciones de cada comando se obtiene con `python aircraft_recon.py COMANDO --help`. El detalle técnico y la organización de las carpetas están en `README.md`.